

DevCore AI — Chronological Runtime Execution Trace

Line-by-Line Runtime Execution Flow from Boot to Blueprint Synthesis & PDF Export

Complete Chronological Program Execution Trace

This document traces the **exact, line-by-line runtime execution** of the DevCore AI application from the moment a user launches the web interface, creates a new project, runs the 13-agent workflow, inspects Mermaid architecture diagrams, generates code inside Implementation Studio, and exports final PDF deliverables.

STEP 1: Application Boot & Session State Initialization

- **1. Responsible File:** [app.py](#)
- **2. Function Name:** `_init_session_state()`
- **3. Responsible Line of Code:** Line 15 (`set_page_config`) & Line 55 (`_init_session_state`)
- **4. Key Code Snippet:**

```
st.set_page_config(page_title="DevCore AI", layout="wide")
_init_session_state()
st.html(get_css())
```

- **5. Input:** Executed automatically when ``streamlit run app.py`` starts.
- **6. Output:** Initialized Streamlit session state keys with default values.
- **7. Next Function Called:** `init_db()` in `database/schema.py`
- **8. Why This Step Is Called:** Establishes app settings and prevents `KeyError` crashes when UI components read state variables.

STEP 2: Database Connection & DDL Schema Bootstrapping

- **1. Responsible File:** [database/schema.py](#)
- **2. Function Name:** `init_db()`
- **3. Responsible Line of Code:** Line 12 (`init_db`) & Line 88 (seed settings)
- **4. Key Code Snippet:**

```
conn = sqlite3.connect('ai_software_company.db')
conn.execute('PRAGMA journal_mode=WAL;')
cursor.execute('CREATE TABLE IF NOT EXISTS projects (...)' )
```

- **5. Input:** SQLite database file path.
- **6. Output:** Created database tables (`projects`, `agent_runs`, `chats`, `settings`) and seeded default settings.
- **7. Next Function Called:** Page Router (`current_page`) in `app.py`
- **8. Why This Step Is Called:** Guarantees database structure exists and seeds `execution_provider = 'groq'`.

STEP 3: New Project Wizard Submission & Validation

- **1. Responsible File:** [pages/new_project.py](#)
- **2. Function Name:** `render_new_project_wizard()`
- **3. Responsible Line of Code:** Line 121 (`INSERT INTO projects`)
- **4. Key Code Snippet:**

```
execute_update('INSERT INTO projects (id, name, description, industry, tech_preference, budget, timeline, difficulty, model_used) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)', (project_id, ...))
st.session_state['created_projects'][project_id] = project_dict
```

- **5. Input:** User submitted form inputs (Project Name, Description, Industry, Stack, Budget).
- **6. Output:** New project record saved in SQLite database and cached in `st.session_state['created_projects']`.
- **7. Next Function Called:** `run_project_planning(project_id)` in `workflow/graph.py`
- **8. Why This Step Is Called:** Persists project record so it is accessible across all workspace pages.

STEP 4: LangGraph Workflow Graph Building

- **1. Responsible File:** [workflow/graph.py](#)
- **2. Function Name:** `build_workflow_graph()`
- **3. Responsible Line of Code:** Line 41 (`build_workflow_graph`)
- **4. Key Code Snippet:**

```
workflow = StateGraph(ProjectState)
for role in roles: workflow.add_node(role, create_agent_node(role))
workflow.set_entry_point('ceo')
compiled_graph = workflow.compile()
```

- **5. Input:** ProjectState schema definition.
- **6. Output:** Compiled StateGraph runnable engine.
- **7. Next Function Called:** `run_project_planning()` in `workflow/graph.py`
- **8. Why This Step Is Called:** Wires 13 specialized engineering personas into a dependency pipeline.

STEP 5: Sequential 13-Agent Execution Pipeline

- **1. Responsible File:** [agents/base.py](#)
- **2. Function Name:** `run_agent()`
- **3. Responsible Line of Code:** Line 145 (`query_ollama`) & Line 159 (`INSERT INTO agent_runs`)
- **4. Key Code Snippet:**

```
output_markdown = query_ollama(system_prompt, user_prompt, agent_role=agent_role)
execute_update('INSERT INTO agent_runs (project_id, agent_role, output_markdown, execution_time_s) VALUES (?, ?, ?, ?)', (project_id, agent_role, output_markdown, elapsed_time))
```

- **5. Input:** `project_id`, `agent_role` (e.g. 'ceo', 'architect', 'backend'), shared state.
- **6. Output:** 8,192 token Markdown specification sheet saved to SQLite `agent_runs` table.
- **7. Next Function Called:** `query_ollama()` -> `query_groq_api_fallback()` in `utils/ollama_client.py`
- **8. Why This Step Is Called:** Synthesizes structured engineering specifications for each persona.

STEP 6: Dynamic LLM Provider & Groq API Key Dispatch

- **1. Responsible File:** [utils/ollama_client.py](#)
- **2. Function Name:** `query_groq_api_fallback()`
- **3. Responsible Line of Code:** Line 88 (key lookup) & Line 125 (`stream_groq_response`)
- **4. Key Code Snippet:**

```
groq_k = st.session_state.get('groq_api_key_blueprint') or key_map.get('groq_api_key_blueprint')
chunks = list(stream_groq_response(messages=msg, api_key=groq_k, model='llama-3.3-70b-versatile'))
```

- **5. Input:** `system_prompt`, `user_prompt`, `agent_role`.
- **6. Output:** Live streamed response text from Groq Cloud API.
- **7. Next Function Called:** Persists output to SQLite in `agents/base.py`
- **8. Why This Step Is Called:** Dispatches queries to Groq Cloud API using dedicated role keys.

STEP 7: Project Workspace Explorer & Session Merging

- **1. Responsible File:** [pages/projects.py](#)
- **2. Function Name:** `show_projects()`
- **3. Responsible Line of Code:** Line 608 (`execute_query`) & Line 613 (`session merge`)
- **4. Key Code Snippet:**

```
db_projects = execute_query('SELECT * FROM projects ORDER BY created_at DESC')
session_projects = list(st.session_state.get('created_projects', {}).values())
project_map = {p['id']: p for p in session_projects + db_projects}
```

- **5. Input:** Active project ID selected from dropdown.
- **6. Output:** Merged list of projects displaying active blueprint spec sheet.
- **7. Next Function Called:** `render_mermaid_in_streamlit()` in `pages/projects.py`
- **8. Why This Step Is Called:** Guarantees new projects created on web UI appear in dropdown instantly.

STEP 8: Interactive Visual Mermaid SVG Iframe Rendering

- **1. Responsible File:** [pages/projects.py](#)
- **2. Function Name:** `render_mermaid_in_streamlit()`
- **3. Responsible Line of Code:** Line 280 (HTML template) & Line 598 (`st.iframe`)
- **4. Key Code Snippet:**

```
html_code = f'...  ...'  
st.iframe('data:text/html;charset=utf-8,' + urllib.parse.quote(html_code), height=height)
```

- **5. Input:** Mermaid diagram code string.
- **6. Output:** Interactive pan/zoom SVG canvas rendered in sandboxed iframe.
- **7. Next Function Called:** `render_implementation_studio()` in `components/implementation_studio.py`
- **8. Why This Step Is Called:** Provides smooth 60fps pan/zoom without causing Streamlit page reruns.

STEP 9: Code Generator Studio Multi-File Synthesis

- **1. Responsible File:** [utils/implementation_engine.py](#)
- **2. Function Name:** `execute_implementation_module()`
- **3. Responsible Line of Code:** Line 109 (`query_fn`) & Line 118 (`sanitize_and_eliminate_placeholders`)
- **4. Key Code Snippet:**

```
raw_output = query_fn(system_prompt, user_prompt, agent_role='studio', override_max_tokens=8192)  
cleaned_text = sanitize_and_eliminate_placeholders(raw_output, proj_name)
```

- **5. Input:** `project_id`, category (e.g. 'Database DDL', 'REST Routers').
- **6. Output:** Multi-file runnable source code package.
- **7. Next Function Called:** `generate_project_pdf_report()` in `utils/pdf_generator.py`
- **8. Why This Step Is Called:** Generates 5 to 10 runnable implementation code files for the active module.

STEP 10: Single-Click PDF Deliverable Compilation

- **1. Responsible File:** [utils/pdf_generator.py](#)
- **2. Function Name:** generate_project_pdf_report()
- **3. Responsible Line of Code:** Line 35 (SimpleDocTemplate) & Line 210 (doc.build)
- **4. Key Code Snippet:**

```
doc = SimpleDocTemplate(buffer, pagesize=letter)
story.append(Paragraph('Technical Deliverable Package', title_style))
doc.build(story, canvasmaker=NumberedCanvas)
```

- **5. Input:** project_details dict, agent_runs list.
- **6. Output:** Binary PDF byte stream delivered to st.download_button.
- **7. Next Function Called:** File Download in User local file system
- **8. Why This Step Is Called:** Compiles all 13 agent blueprints into a single offline PDF manual.

Linear Program Execution Map

```
USER SUBMITS NEW PROJECT
■
▼
[app.py] ■■> _init_session_state() & get_css()
■
▼
[database/schema.py] ■■> init_db() creates tables & seeds execution_provider = 'groq'
■
▼
[pages/new_project.py] ■■> INSERT INTO projects & caches in st.session_state['created_projects']
■
▼
[workflow/graph.py] ■■> build_workflow_graph() compiles 13-node StateGraph
■
■■■ [agents/base.py] ■■> CEO Node -> query_ollama(agent_role='ceo')
■■■ [utils/ollama_client.py] ■■> query_groq_api_fallback() queries Groq Cloud API
■■■ [agents/base.py] ■■> Saves Markdown output to 'agent_runs' table
■
■■■ Sequentially runs BA -> PM -> Architect -> UI/UX -> Frontend -> Backend -> ... -> Reviewer
■
▼
[pages/projects.py] ■■> show_projects() merges session state & renders Workspace Explorer
■
■■■ [pages/projects.py] ■■> render_mermaid_in_streamlit() renders interactive pan/zoom SVG
■■■ [components/implementation_studio.py] ■■> execute_implementation_module() generates source code
■
▼
[utils/pdf_generator.py] ■■> generate_project_pdf_report() compiles ReportLab PDF deliverable
■
▼
[app.py] ■■> st.download_button() delivers complete PDF manual to user's disk
```